

Ganzheitliches Technisches Umsetzungskonzept für die Systemarchitektur (Maximal Erweitert)

Umfassender Architekturleitfaden zur Implementierung von verteilter Datenkonsistenz, erweiterten Geschäftsfunktionen, modularer Resilienz, Multi-Views und Simulations-Suiten

Dokumenten-Typ: Erweitertes Technisches Umsetzungskonzept (Architekturleitfaden)

Entwicklungsstand: Vollständige Abdeckung aller Pflicht-, Business- und Bonuskomponenten

Technologiebasis: Node.js, TypeScript, NestJS, PostgreSQL, RabbitMQ, Redis, React

Zielgruppe: Entwicklungsteam & Lehrstuhl für Software Engineering (SoSe 2026)

1. SYSTEMLANDSCHAFT & MODULVERANTWORTLICHKEITEN

Das zu entwickelnde System bildet die technische Infrastruktur eines microservice-orientierten Online-Shops ab. Um eine minimale Kopplung und maximale Wartbarkeit zu garantieren, wird eine strikte fachliche Schichttrennung angewendet. Jeder Microservice ist für seinen eigenen Datenbereich zuständig und besitzt eine vollständig isolierte Datenbankinstanz. Ein direkter Zugriff auf fremde Datenbanken ist datenarchitektonisch verboten.

1.1 Übersicht der Systemkomponenten

Die Anwendungslandschaft setzt sich aus fünf eigenständigen Services zusammen:

Service	Fachliche Verantwortlichkeit	Datenhaltung & Infrastruktur
Shop-Service	Produktkatalog, Warenkorb, Bestellauslösung. Agiert als einziges API-Gateway nach außen und übernimmt die Rolle des Saga-Orchestrators . Kapselt zudem das vollständige Benutzer- und Berechtigungsmanagement.	PostgreSQL (Bestellungen, Produkte, Nutzer) & Redis (Idempotenz & Sessions)
Warehouse-Service	Pseudo-Warenwirtschaft: prüft Lagerbestand, reserviert und bucht Ware aus. Bietet administrative Schnittstellen zur Einbuchung neuer Warenlieferungen. Kann gezielt Fehler werfen.	PostgreSQL (Bestände mit Zeilensperren)
Billing-Service	Orchestriert den Zahlungsfluss. Kommuniziert ausschließlich über eine Payment-Fassade mit externen Anbietern.	PostgreSQL (Zahlungslogs & Transaktionsstatus)
Invoice-Service	Erzeugt PDF-Rechnungen und speichert diese. Wird nach erfolgreicher Zahlung asynchron aufgerufen.	Dateisystem / Volume (PDF-Speicher)
Audit-Service	Empfängt Domain-Events aller Services, persistiert unveränderliche Audit-Snapshots in der Datenbank. Darf kein Business-Wissen besitzen.	PostgreSQL (Append-Only Snapshot-Tabelle)

1.2 Kommunikationsmatrix

Die Services nutzen zwei komplementäre Kommunikationsparadigmen:

- **Synchron (REST/HTTP):** Shop-Service zu Warehouse-Service, Shop-Service zu Billing-Service.
- **Asynchron (Message Queue):** Billing-Service zu Invoice-Service (PaymentSucceeded-Event), alle Services zu Audit-Service (Domain-Events). Implementiert über RabbitMQ.

1.3 Detaillierte Microservice-Spezifikation & Endpunkte

Jeder Service exponiert spezifische, ressourcenorientierte REST-Endpunkte. Alle API-Antworten im Fehlerfall erfolgen strikt nach RFC 7807 (Problem Details for HTTP APIs).

Shop-Service (Gateway, Core Business & Identity Access Management)

Dient als einziger Eintrittspunkt für die Frontends. Kapselt neben der Saga-Orchestration das vollständige E-Commerce-Kerngeschäft sowie die Sicherheitsarchitektur.

- **Nutzer- und Rechteverwaltung:**
 - `POST /v1/auth/register` : Registriert neue Benutzer im System.

- `POST /v1/auth/login` : Authentifiziert Nutzer und stellt JWT-Tokens aus, welche die Benutzerrolle (`CUSTOMER` , `ADMIN` , `LOGISTICS`) im Payload verschlüsseln.

• Produktmanagement:

- `GET /v1/products` : Öffentlicher Endpunkt zur Produktübersicht im Katalog. Unterstützt Paginierung und Filterung.
- `POST /v1/products` : Erstellt ein neues Produkt im Katalog. Dieser Endpunkt ist über Guards strikt auf die Rolle `ADMIN` beschränkt. Schickt nach Erfolg ein synchrones Signal an den Warehouse-Service zur Initialisierung des Bestandsdatensatzes.

• Warenkorbmanagement (Basket Management):

- `GET /v1/baskets` : Ruft den aktiven Warenkorb des authentifizierten Nutzers ab (Persistierung in der PostgreSQL-Datenbank des Shop-Services).
- `POST /v1/baskets/items` : Fügt Artikel dem Warenkorb hinzu oder erhöht deren Anzahl.
- `DELETE /v1/baskets/items/{productId}` : Entfernt eine Position restlos aus dem Warenkorb.

• Bestellauslösung:

- `POST /v1/orders` : Transformation des Warenkorbs in eine Bestellung. Erfordert die Rolle `CUSTOMER` , validiert die Verfügbarkeit, liest den Header `X-Idempotency-Key` aus und startet den verteilten Saga-Prozess.
- `GET /v1/orders/{correlationId}` : Liefert den aktuellen Verarbeitungsstatus.

Warehouse-Service (Bestandsführung & Logistik-Schnittstellen)

Verwaltet die physischen oder digitalen Lagerbestände und sichert diese gegen konkurrierende Zugriffe ab. Bietet logistische Endpunkte für Warenlieferungen.

- `POST /v1/warehouse/reservations` : Prüft den Bestand exklusiv und legt eine temporäre Reservierung an.
- `DELETE /v1/warehouse/reservations/{correlationId}` : Kompensations-Endpunkt. Storniert die Reservierung bei abgelehnter Zahlung.
- `POST /v1/warehouse/bookings` : Bucht die reservierten Artikel endgültig aus. Kann künstlich Fehler werfen (für Simulationen).

• Warenwirtschaft & Logistik (Produktbestände editieren):

- `PATCH /v1/warehouse/products/{productId}/stock` : Ermöglicht es Logistik-Mitarbeitern (Rolle `LOGISTICS`), den physischen Bestand anzupassen, wenn Lieferungen im Lager eingehen. Erhöht den verfügbaren Bestand im System atomar.

Billing-Service (Zahlungsabwicklung & Fassadensteuerung)

Kapselt die Anbieter-Stubs und hält die Zahlungszustände vor.

- `POST /v1/billing/charges` : Initiiert den Einzug über die aktive Payment-Fassade.
- `POST /v1/billing/refunds` : Kompensations-Endpunkt. Führt einen Refund über die Fassade aus.
- `POST /v1/billing/webhooks/{provider}` : Empfängt asynchrone Zahlungsbestätigungen der simulierten Webhook-Stubs.

Invoice-Service (Dokumentengenerierung)

Verfügt über **keine öffentlichen REST-Endpunkte**. Er lauscht rein asynchron auf den RabbitMQ-Broker, verarbeitet das `PaymentSucceeded` -Event und legt die generierte PDF ab.

Audit-Service (Event-Store)

Verwaltet die Snapshots. Muss für das Admin-Dashboard lesbar sein.

- `GET /v1/audit/orders/{correlationId}` : Liefert alle Snapshots einer Bestellung chronologisch sortiert.
- `GET /v1/audit/stream` : SSE-Endpunkt für die Echtzeit-Aktualisierung des Admin-Dashboards.

2. TRANSAKTIONSSTEUERUNG VIA ORCHESTRIERTES SAGA-MUSTER

Schlägt ein Service-Aufruf fehl, wird der gesamte Vorgang konsistent über ein Saga-Muster zurückgerollt. Der Shop-Service übernimmt die Orchestrierung.

2.1 Synchroner Ablauf im Erfolgsfall (Happy Path)

Ein erfolgreicher Bestellvorgang durchläuft die folgenden Phasen:

1. **Bestellung einleiten:** Shop-Service empfängt `POST /orders`, validiert die Anfrage und weist eine eindeutige `correlationId` zu.
2. **Lager prüfen & reservieren:** Shop-Service ruft synchron den Warehouse-Service auf. Dieser prüft Bestände und legt eine Reservierung an.
3. **Zahlung initiieren:** Billing-Service wird aufgerufen. Er delegiert über die Payment-Fassade an den konfigurierten Anbieter.
4. **Zahlung bestätigen:** Zahlungsanbieter-Stub gibt Erfolg zurück. Billing-Service protokolliert das Ergebnis.
5. **Rechnungserstellung:** Billing-Service publiziert ein `PaymentSucceeded`-Event. Invoice-Service erstellt asynchron eine PDF-Rechnung.
6. **Lager ausbuchen:** Warehouse-Service bucht die reservierten Artikel endgültig aus.
7. **Bestellung abschließen:** Shop-Service setzt den Status auf `COMPLETED` und antwortet dem Client.

2.2 Fehlerszenarien und Saga-Kompensation

Das System muss alle bereits ausgeführten Schritte rückgängig machen (Kompensationstransaktionen). Jeder Kompensationsschritt muss als eigener Audit-Snapshot gespeichert werden.

Szenario A: Zahlung abgelehnt

Die Reservierung im Warehouse-Service wird storniert. Die Bestellung erhält den Status `PAYMENT_FAILED`.

Szenario B: Lager nicht ausreichend

Es erfolgt kein weiterer Aufruf; die Bestellung erhält den Status `OUT_OF_STOCK`.

Szenario C: Warehouse-Ausbuchung schlägt fehl

Die Zahlung wird rückerstattet (Refund über Fassade). Die Bestellung erhält den Status `ROLLBACK_COMPLETED`.

3. DATENKONSISTENZ & CONCURRENCY CONTROL IM WAREHOUSE

Bei hoher Last (z. B. Flash-Sales) muss der Warehouse-Service garantieren, dass Bestandsprüfungen absolut konsistent sind, um den Fehler "Lager nicht ausreichend" korrekt zu determinieren und Überverkäufe zu verhindern.

3.1 Pessimistic Locking via Datenbank

Der Warehouse-Service kapselt die Bestandsprüfung und die Reservierung innerhalb einer isolierten Transaktion. Die betroffene Zeile in der Produkttabelle wird mittels `SELECT ... FOR UPDATE` für alle anderen Transaktionen gesperrt. Konkurrierende Threads werden von PostgreSQL in einen Wartezustand versetzt, bis die Transaktion mittels Commit oder Rollback beendet wird. Ein Dirty-Read oder Überverkauf ist damit physikalisch ausgeschlossen.

4. PAYMENT-FASSADE & SIMULATIONSPAKETE

Die Payment-Fassade ist das architektonische Herzstück und wird im Billing-Service implementiert.

4.1 Architektur der Fassade

- **Einheitliche Schnittstelle:** Alle Aufrufe laufen ausschließlich über die Fassade; kein direkter Zugriff auf einen konkreten Anbieter.
- **Konfigurierbarkeit:** Der aktive Anbieter wird per Umgebungsvariable gesetzt, nicht per Code.
- **Erweiterbarkeit:** Ein dritter Anbieter muss hinzufügbare sein, ohne bestehenden Code zu verändern (Open/Closed Principle).
- **Operationen:** Muss zwingend `charge`, `refund` und `getStatus` implementieren.

4.2 Asynchroner Webhook-Anbieter

Ein Anbieter-Stub wird erweitert, um asynchrone Zahlungen zu simulieren. `charge()` antwortet sofort mit status: `PENDING`. Nach konfigurierbarer Verzögerung sendet der Stub einen Webhook-Request an den Billing-Service, der Erfolg oder Misserfolg signalisiert. Der Billing-Service führt in diesem Modus den Saga-Ablauf korrekt weiter.

4.3 Eingesetzte Simulationspakete & Bibliotheken

Da für dieses rein simulatorische Universitätsprojekt keine echten API-Schlüssel oder externen Bezahl Dienste angebunden werden dürfen, kommen spezifische Simulationsbibliotheken im Billing-Service zum Einsatz:

- **MSW (Mock Service Worker):** Wird verwendet, um ausgehende HTTP-Anfragen der Zahlungsanbieter auf Netzwerkebene abzufangen. Dadurch verhalten sich die Adapter im Code absolut realistisch, rufen jedoch einen lokalen MSW-Handler auf, der die API-Dokumentation von Stripe oder Klarna emuliert.
- **Faker.js (@faker-js/faker):** Generiert künstliche, aber strukturell valide Transaktions-IDs, Autorisierungs-Tokens und Fehlermeldungen innerhalb der Stubs, um realistische Log-Daten und Audit-Payloads zu erzeugen.

5. AUDIT, IDEMPOTENZ & RESILIENZ

5.1 Audit- und Snapshot-System

Jeder Zustandsübergang wird als unveränderlicher Datensatz gespeichert (Event Sourcing Light). Snapshots sind nach dem Einfügen unveränderlich (keine UPDATE- oder DELETE-Operationen). Das Schema umfasst zwingend `correlationId`, `eventType`, `timestamp`, `payload` (als JSON) und `previousEventId` (als Verkettung). Ein REST-Endpoint liefert die Historie.

5.2 Idempotenz-Garantien

Der Endpoint `POST /orders` ist idempotent. Sendet ein Client denselben Request zweimal (identischer Idempotency-Key-Header), darf die Bestellung nur einmal ausgeführt werden. Der zweite Request muss dasselbe Ergebnis liefern, ohne die Daten erneut zu verändern. Dies wird über eine Redis-Cache-Middleware im Shop-Service realisiert.

5.3 Circuit Breaker am Invoice-Service

Ist der Invoice-Service nicht erreichbar, wird die Zahlung nicht rückgängig gemacht. Stattdessen wird ein Circuit Breaker (z.B. Opossum) implementiert. Nach drei aufeinanderfolgenden Fehlern wechselt er in den Open-Zustand und lässt nach 30 Sekunden einen Test-Request zu (Half-Open). Zustandswechsel werden als Audit-Snapshot persistiert. Ein Retry-Mechanismus (mind. 3 Versuche) ist ebenfalls integriert.

6. FRONTEND-ARCHITEKTUR & SIMULATIONS-SUITES

Um die geforderte Systemtransparenz, die Echtzeit-Visualisierungen und die zwingend notwendige "Live-Demo: Bestellprozess Happy Path und mind. ein Fehlerszenario" während der Präsentation professionell abzubilden, werden dedizierte React-Frontend-Ansichten (Views) entwickelt.

6.1 Customer View (E-Commerce Storefront)

Das kundenorientierte Frontend. Simuliert das Einkaufserlebnis.

- **Funktionalität:** Artikelauswahl, interaktives Warenkorbmanagement und Kassen-Dialog. Benutzer können sich registrieren, einloggen und ihren spezifischen Warenkorb live befüllen.
- **Architektonischer Wert:** Fängt Idempotency-Szenarien ab (z.B. durch Doppel-Klicks auf "Kaufen" bei simulierter Netzwerklatenz).
- **Visualisierung:** Wandelt den asynchronen Saga-Ablauf in eine nutzerfreundliche Loading-UI um (z. B. "Zahlung wird autorisiert...").

6.2 Admin View (Admin-Dashboard)

Erfüllt die Anforderungen aus Bonusaufgabe 4.3.

- **Übersicht:** Listet alle Bestellungen mit aktuellem Status. Filterung nach Bestellstatus, Zeitraum und correlationId. Bietet administrativen Zugriff zur Produkterstellung (Rolle `ADMIN`).
- **Detailansicht:** Vollständige Audit-Timeline als Schritt-für-Schritt-Ablauf. Echtzeit-Aktualisierung per Server-Sent Events oder WebSocket.
- **Deep-Dive:** Erlaubt das Ausklappen des JSON- `payloads` jedes Audit-Snapshots zur technischen Fehleranalyse.

6.3 Warehouse Logistics View (WMS)

Spezialisierte Ansicht für die Lagerverwaltung, um den Systemzustand transparent zu machen.

- **Bestandsmonitor:** Zeigt Live-Daten zu "Freiem Bestand" und "Reserviertem Bestand" an.
- **Wareneingangssteuerung:** Ermöglicht es Mitarbeitern (Rolle `LOGISTICS`), über eine Eingabemaske neue Warenlieferungen zu erfassen und die Bestände über die `PATCH`-Schnittstelle im Warehouse-Service live zu editieren.
- **Transparenz:** Veranschaulicht, wie eine Bestellung den Bestand zunächst temporär reserviert und erst nach dem `PaymentSucceeded`-Event physisch abbucht.

6.4 Simulations-Frontend (Architektur-Stresstest & Demo-Tool)

Ein isoliertes Control-Panel, das exklusiv für die Live-Präsentation und das System-Debugging entwickelt wird. Da der Warehouse-Service gezielt Fehler werfen können muss, steuert dieses Interface die Verhaltensweisen der Backend-Komponenten:

- **Künstlicher Happy Path Trigger:** Löst automatisierte Muster-Bestellungen aus, um den Log-Stack und das Dashboard für die Demo mit Daten zu befüllen.
- **Error Provocation (Fehler-Injektion):**
 - Schalter 1: Zwingt den Payment-Stub, beim nächsten Aufruf eine Ablehnung zu simulieren (Testet Kompensation Szenario A).
 - Schalter 2: Blockiert das endgültige Ausbuchen im Warehouse (Testet Kompensation Szenario C).
 - Schalter 3: Zwingt den Invoice-Service zum Absturz (Testet Circuit-Breaker Öffnung und Retries).
- **Concurrency Provocation (Nebenläufigkeitstest):** Ein dedizierter Button, der exakt denselben Artikel über zwei parallele Threads in Millisekunden-Abständen anfragt. Er demonstriert live das Greifen des Pessimistic Lockings im Warehouse-Service und erzwingt das saubere Abweisen des zweiten Requests via `OUT_OF_STOCK` -Saga-Pfad.
- **Webhook-Latenz-Slider:** Reguliert die künstliche Verzögerung des Webhook-Stubs, um asynchrone Zahlungen im Admin-Dashboard visuell erlebbar zu machen.

7. ZENTRALES LOG-MANAGEMENT & TRACING

Jeder Service führt ein strukturiertes Log im JSON-Format. Jeder Logeintrag enthält `correlationId`, `service`, `level`, `timestamp`, `message` und `context`. Der Header `X-Correlation-Id` wird von jedem Service ausgelesen und angehängt.

Zur Erfüllung von Bonus 4.5 wird ein zentraler Log-Stack (Loki/Grafana) angebunden. Das Grafana-Dashboard visualisiert mindestens:

- Anzahl der Bestellungen pro Zeitintervall.
- Fehlerrate nach Service.
- Zeitreihe der Zahlungsversuche nach Ergebnis (Erfolg / Ablehnung / Timeout).

8. INFRASTRUKTUR & DEPLOYMENT

Alle Services werden zustandslos entworfen. Deployment via Docker Compose ist Pflicht; alle Services müssen per `docker-compose up` startbar sein. Konfiguration erfolgt ausschließlich über Umgebungsvariablen - keine hartcodierten Werte im Code. Die Infrastruktur skaliert kritische Services (Warehouse) via Docker Compose Replicas horizontal, inkl. DNS-Load-Balancing.

9. REPOSITORY-STRUKTUR & DOKUMENTATION

Das Git-Repository spiegelt die Vorgaben der Architektur exakt wider:

```
/
├─ docker-compose.yml
├─ docs/
│   └─ architecture.md           # Kontext-, Komponenten-, Sequenzdiagramme
│   └─ decisions/               # ADRs: Persistenz, Message Broker, Frontend
├─ shop-service/               # Gateway & Saga Orchestrator (Katalog, Warenkorb, IAM)
├─ billing-service/            # Kapselt Payment-Fassade (MSW / Faker Integration)
├─ warehouse-service/          # WWS & Pessimistic Locking (Logistik-Schnittstellen)
├─ invoice-service/            # Asynchrone PDF-Erstellung (Circuit Breaker)
├─ audit-service/              # Append-Only Event Store
└─ frontends/
    ├─ storefront-ui/          # Customer View
    ├─ admin-dashboard/         # Admin View & Produkterstellung (Bonus 4.3)
    ├─ warehouse-wms/           # Warehouse Logistics View & Wareneingang
    └─ simulation-panel/        # Demo-, Fehler- & Concurrency-Simulations-Suite
```